

Laboratorio de Programación y Estructuras de Datos (PRD) Examen de laboratorio (Grupo 11) – Duración: 1:30 h

En este examen de laboratorio de PRD se van a implementar las estructuras de datos necesarias para gestionar unas elecciones, tal y como se detalla a continuación. Recordad que el código entregado tiene que compilar para poder aprobar este examen. Disponéis de las siguientes estructuras de datos y funciones definidas en lenguaje C, cuya declaración se incluye en los ficheros de cabecera voto_pila.h y elector.h:

```
/* voto_pila.h */
#define VOT_SI      1
#define VOT_NO      0
#define VOT_BLANCO -1
#define NUM_VOT     20

typedef struct voto{
    int valor;
    int hora;
    struct voto *next;
}t_voto;

typedef struct{
    int nvotos;
    t_voto *top;
}t_votos;

int inicializar_votos (t_votos **l_votos);          /* HECHA */
int obtener_hora_actual();                          /* HECHA */
int votar (t_votos *l_votos, int valor);           /* HECHA */
void mostrar_votos (t_votos *l_votos);             /* HECHA */
int liberar_votos (t_votos **l_votos);             /* HECHA */

/* elector.h */
#define MAX_NOM 20
#define MAX_EL  200
#define SI      1
#define NO      0

typedef struct
{
    int numero;      /* Número del NIF */
    char lletra;     /* Letra del NIF */
}t_nif;

typedef struct
{
    t_nif nif;
    char nombre[MAX_NOM], apellido1[MAX_NOM], apellido2[MAX_NOM];
    int ha_votado;    /* Valores posibles SI, NO */
}t_elector;

typedef struct
{
    int numelec;
    t_elector electores[MAX_EL];
}t_electorado;

int inicializar_electorado (t_electorado *l_elect);          /* HECHA */
int buscar_elector (t_electorado *l_elect, t_nif n);
int anyadir_elector (t_electorado *l_elect, t_elector e);
int votar_elector (t_electorado *l_elect, t_votos *v, t_nif n, int voto);
int comprobar_voto (t_electorado *l_elect, t_nif n);
void mostrar_electorado (t_electorado *l_elect);            /* HECHA */
```

1. Apartados

Se pide implementar las funciones indicadas en los siguientes apartados e invocarlas desde el programa principal. Algunas funciones ya están implementadas y sólo las tenéis que utilizar.

- a) **(1 punto)** La función `buscar_elector()` debe comprobar si un elector está entre el electorado. Si está, debe devolver la posición. Si no está, debe devolver -1.
- b) **(2 puntos)** La función `anyadir_elector()` debe añadir un nuevo elector al final de la lista de electores. Si la lista está llena o el elector ya estaba en la lista, debe devolver -1. En caso de que se pueda añadir el elector, debe devolver 0.
- c) **(3 puntos)** La función `votar_elector()` debe registrar el voto realizado por el elector identificado por el parámetro `n`. Para ello, debe comprobar que hay un elector con el nif `n`. Si el elector no está entre los electores, debe devolver -1. Si el elector está, se debe llamar a la función `votar()`, para que se añada el voto a la pila de votos del parámetro `v`. Si la función `votar` es correcta, se debe devolver 0. Si la función `votar` da algún error, se debe devolver -1.
- d) **(1 punto)** La función `comprobar_voto()` debe comprobar si un elector ha votado ya o no. Para ello, debe comprobar que hay un elector con el nif `n`. Si el elector no está entre los electores, debe devolver -1. Si el elector está, devolverá 1 si ha votado ya y 0 en caso contrario (no ha votado todavía).
- e) **(2 puntos)** El programa principal debe realizar las siguientes operaciones: inicializar las variables que representan al electorado y los votos. Añadir tres electores correctamente y mostrar el electorado. A continuación, debe votar el primer elector 2 veces (la segunda debería dar error) y una vez el segundo elector. Se debe mostrar el electorado de nuevo y después los votos actuales. Se debe comprobar el resultado de cada operación para poder continuar con la ejecución del programa. En caso de error en alguna de las funciones, el programa debe finalizar correctamente.
- f) **(1 punto)** Utilice la herramienta `valgrind` para comprobar que su programa no tiene pérdidas de memoria. A partir de los resultados de `valgrind`, ¿cuántos bloques de memoria dinámica se han reservado y liberado durante la ejecución del programa desarrollado en este examen? ¿De qué tamaño (en Bytes) son los bloques de memoria? ¿Por qué razón son de ese tamaño? Añada sus respuestas a estas preguntas como comentario en el mismo fichero `main.c`, justo después de la implementación de la función `main()`.

Ejemplo de ejecución:

Elector añadido Elector añadido Elector añadido Electorado: Nombre y apellidos: Pepito Perez Perez NIF: 11111111-H ¿Ha votado? NO Nombre y apellidos: Juanito Garcia Garcia NIF: 22222222-D ¿Ha votado? NO Nombre y apellidos: Jorgito Lopez Lopez NIF: 33333333-I ¿Ha votado? NO Voto correcto Error votando Voto correcto	Electorado: Nombre y apellidos: Pepito Perez Perez NIF: 11111111-H ¿Ha votado? SI Nombre y apellidos: Juanito Garcia Garcia NIF: 22222222-D ¿Ha votado? SI Nombre y apellidos: Jorgito Lopez Lopez NIF: 33333333-I ¿Ha votado? NO Voto: 0 Valor: En blanco Emitido a las 18:40 Voto: 1 Valor: NO Emitido a las 18:35
--	--